WaferAI · Fab AI Inspector

개발 진행 보고서

WM-811K 웨이퍼 결함 분류 모델의 웹 서비스 배포
Phase 0 – 4 완료 · Docker 배포(Phase 5) 준비 단계

항목	내용
프로젝트	WaferAI (Fab AI Inspector)
팀	팀 Wafer-Eye · 조원곤, 변지민
소속	ICT융합공학부 캡스톤디자인
보고 범위	2학기 배포 작업 Phase 0 – 4
작성일	2026년 9월 24일
모델	CoAtNet-0 (timm coatnet_0_rw_224.sw_in1k), 9클래스
검증 정확도	96.4538% (검증 분할 10,208건)

1. 개요

1.1 목표

1학기에 학습한 CoAtNet-0 웨이퍼 결함 분류 모델을, 다른 사람이 접속해 사용할 수 있는 웹 서비스로 배포하는 것이 이번 학기 목표다. 모델 자체는 재학습하지 않고, 학습 당시의 추론 결과를 그대로 재현하는 서빙 경로를 만드는 것이 전제 조건이었다.

대상 사용자는 중소 반도체 기업의 품질 엔지니어로 상정했다. 따라서 별도 데이터 변환 없이 흔히 가지고 있는 형태의 파일을 그대로 올릴 수 있어야 하고, AI 판정을 그대로 믿게 하는 것이 아니라 근거를 보여주고 사람이 확정하는 구조여야 한다.

1.2 기술 스택

구분	기술	비고
추론	ONNX Runtime (CPU) / PyTorch	백엔드 전환 가능, 두 경로 일치 검증
설명가능성	Grad-CAM (직접 구현)	PyTorch 지연 로딩
API	FastAPI + Uvicorn	JWT httpOnly 쿠키 인증
데이터베이스	SQLite(개발) / PostgreSQL(운영)	SQLAlchemy + Alembic
화면	React 19 + Vite + Tailwind 4	React Router 7
배포	Docker Compose + Nginx	Phase 5 예정

1.3 진행 현황

단계	내용	상태
Phase 0	리포지토리 정리, 의존성 고정	완료
Phase 1	공용 전처리 · 추론 클래스 · /predict · ONNX 익스포트	완료
Phase 1.5	LSWMD 실데이터 성능 측정	완료
Phase 2	PostgreSQL 스키마 · 인증 · 권한 · 임계값 설정	완료
Phase 3	Grad-CAM · 검토 워크플로	완료
Phase 4	React 화면 (로그인 · 대시보드 · 대기열 · 설정)	완료
Phase 5	Docker Compose · Nginx · 서버 배포	대기 (서버 미정)
Phase 6	재학습 루프 (정답 500건 트리거)	미착수

자동화 테스트 126개가 모두 통과한다. 커밋 7개로 정리되어 있다.

2. 핵심 성과

2.1 전처리 재현 — 학습과 소수점까지 일치

서빙에서 전처리가 학습과 한 단계라도 어긋나면 정확도가 조용히 무너진다. 이를 막기 위해 전처리를 하나의 모듈로 모으고, 학습 코드의 검증 경로와 수치가 같은지 검증했다.

단계	내용
① 정사각 패딩	0으로 채움. 나누어떨어지지 않는 1픽셀은 하단·우측
② 리사이즈	224×224, INTER_NEAREST. 반드시 색 변환 전 라벨맵 상태에서
③ 컬러 LUT	0→검정, 1→초록(0,255,0), 2→노랑(255,255,0), RGB 순서
④ 정규화	÷255 후 ImageNet 평균·표준편차

검증 결과 — 검증 분할 10,208건에서 정확도 96.4538%, 학습 기록과 차이 0.0000%p

torchvision 기준 구현과 rtol=0, atol=0 으로 비교해 비트 단위 일치를 확인했고, LSWMD 원본에서 학습 분할을 시드까지 재현해(균형 데이터 51,038건이 학습 로그와 일치) 실데이터로도 대조했다. CPU와 Apple GPU(MPS) 양쪽에서 같은 값이 나왔다.

2.2 이미지 업로드 지원

모델은 사진이 아니라 전기 검사 결과인 0/1/2 웨이퍼 맵으로 학습됐다. 중소기업이 쉽게 구할 수 있는 형태로 받기 위해, 학습과 같은 색으로 렌더링된 맵 이미지(PNG/JPG)를 색 역변환으로 받아들이는 경로를 추가했다.

경로	형식	특성
데이터	.npy / .csv / .json	정확한 경로. 최대 512×512
이미지	.png / .jpg	색 역변환으로 맵 복원. 224×224 권장

실데이터 300건으로 두 경로를 비교한 결과 예측이 99.00% 일치했고 정확도는 0.66%p 낮았다. 달라진 3건은 모두 Loc 관련으로, 원래 경계가 모호한 클래스에서만 흔들렸다. 다른 색 체계로 그려진 이미지나 웨이퍼 실물 사진은 조용히 잘못 해석하지 않고 거부한다.

2.3 판정 로직 오류 발견 및 수정

모델이 9개 클래스 확률을 내놓은 뒤, 자동 통과시킬지 엔지니어에게 보낼지 정하는 규칙이 Human-in-the-Loop의 핵심이다. 그런데 기존 설계 문서 세 곳(기능상세설명서 §3.3.2, AI파이프라인 플로우차트 §2.2, 기존 코드)이 서로 다른 규칙을 말하고 있었고, 기능상세설명서 쪽에는 어느 분기로도 가지 않는 구간이 있어 그대로는 구현이 불가능했다.

이를 4단계로 통일한 뒤 실데이터로 검증하는 과정에서, 문서가 지시한 규칙 하나가 오히려 정확도를 떨어뜨린다는 사실을 발견했다.

> **문서 설계 「결함 밀도 5% 미만이면 정상(none) 확정」이 실제 결함을 덮고 있었다**
>
> 검증 분할 10,208건 측정 결과, 이 규칙에 걸린 1,087건 가운데 291건이 실제로는 결함이었다(Loc 118 · Edge-Ring 54 · Scratch 50 · Edge-Loc 38 · Center 31). 같은 건들을 AI에 맡기면 98.34%로 맞힌다. 결함 다이가 5% 미만이어도 얇은 Scratch나 작은 Loc은 명백한 결함인데, 픽셀 수를 세는 규칙은 패턴을 보지 못하기 때문이다. 임계값을 0.01까지 낮춰도 규칙 정확도가 9.1%에 그쳐, 조정이 아니라 규칙 자체가 틀린 것으로 판단했다.

항목	규칙 적용 시 (문서 설계)	규칙 제거 후 (현재)
전체 정확도	93.78%	96.45%
잘못 통과된 건	430건 (4.21%)	143건 (1.40%)
무인 통과 정확도	95.58%	98.53%
엔지니어 검토 부담	4.67%	4.95%

검토 부담이 0.28%p 늘어나는 대신 잘못 통과되는 건수가 3분의 1로 줄었다. 해당 규칙은 기본값을 끄되 설정으로 되살릴 수 있게 두었고, 설정 화면에는 위 측정 근거를 경고 문구로 띄워 나중에 무심코 켜지 않도록 했다.

반면 고밀도 규칙(밀도 80% 초과 시 Near-full)은 유지했다. 해당 구간 20건이 모두 실제 Near-full이었고 AI도 같은 답을 냈다. 측정상 이득은 없지만, 향후 재학습으로 모델이 드리프트해도 거의 전면 불량인 웨이퍼는 확정적으로 잡아주는 방어선이 된다.

2.4 추론 백엔드 — 측정 후 선택하는 구조

목표 스택은 ONNX Runtime CPU 추론이었다. 익스포트 후 PyTorch와 확률을 대조해 최대 오차 1.79e-07로 동일함을 확인했다. 다만 속도를 실측한 결과 예상과 반대였다.

백엔드	중앙값 (Apple Silicon arm64)
PyTorch	22.5 ms
ONNX Runtime (최적 설정)	31.1 ms

이는 개발 장비(ARM) 기준이며, 배포 대상인 x86 리눅스에서는 ONNX Runtime이 앞서는 경우가 많다. 따라서 어느 한쪽을 확정하지 않고 두 백엔드를 같은 인터페이스 뒤에 두었다. 서버가 정해지면 벤치마크 스크립트를 한 번 돌려 설정 한 줄로 선택한다. 두 경로가 조용히 갈라지지 않도록 확률 일치를 테스트로 고정했다.

2.5 Grad-CAM 설명가능성

AI가 어디를 보고 판단했는지 히트맵으로 보여준다. 기능상세설명서는 pytorch-grad-cam 라이브러리와 어텐션 토큰 변환(reshape_transform)을 지시했으나, 실측 결과 CoAtNet 마지막 스테이지 출력이 (1, 768, 7, 7) 4차원 NCHW 형태여서 해당 변환이 불필요했다. 남는 계산이 30줄 남짓이라 배포 이미지에 의존성을 더하지 않고 직접 구현했다.

히트맵은 값이 나오는 것만으로는 부족하다. 위치가 엉뚱하면 설명가능성이 아니라 잘못된 확신을 주기 때문이다. 합성 패턴으로 히트맵 질량 중심의 반경을 측정해 검증했다.

합성 패턴	히트맵 질량 반경	기대 위치
Center	0.405	가운데
Donut	0.547	중간 고리
Edge-Ring	0.880	가장자리

결함 기하와 순서가 정확히 일치한다. 이 관계를 테스트로 고정했다.

히트맵은 지연 생성한다. 실제로 열어보는 것은 검토 대상 5% 남짓이므로 전부 미리 만들면 서버가 낭비된다. PyTorch 모델 자체도 첫 히트맵 요청에서야 올라가므로, 아무도 열지 않으면 102MB를 적재하지 않는다. 이는 메모리가 넉넉하지 않은 저가 서버에서 유효한 절약이다.

2.6 인증과 권한

로그인 토큰을 httpOnly 쿠키에 담는다. localStorage에 두면 XSS 한 번에 탈취되지만 httpOnly 쿠키는 자바스크립트가 읽지 못한다. 등급은 3단계이며, 화면에서 버튼을 감추는 것은 방어가 아니므로 모든 엔드포인트가 서버에서 등급을 확인한다.

등급	조회·검토	임계값 변경	계정 관리
engineer	O	X	X
senior_engineer	O	O	X
admin	O	O	O

비밀번호는 bcrypt로 해싱하되 SHA-256으로 길이를 먼저 고정한다. bcrypt는 72바이트에서 입력을 자르는데, 한글은 글자당 3바이트라 24자면 한도에 닿고 잘린 뒤쪽은 검증에 쓰이지 않아 비밀번호가 사실상 짧아지기 때문이다. 로그인 실패 응답은 사번이 없을 때와 비밀번호가 틀릴 때를 메시지와 응답 시간 모두에서 구분하지 않아 사번 열거를 막는다.

ERD 문서에는 사용자 테이블이 없었다. UIUX 문서 화면 8이 사번·부서·권한을 요구하는데 검토자 필드가 자유 문자열이라 누가 검토했는지 추적할 수 없었으므로, users 테이블을 추가하고 외래키로 연결했다.

2.7 화면

로그인 후 메인 대시보드가 나오고, 헤더 아래 탭으로 다른 화면으로 이동한다. 등급에 따라 탭 구성이 달라진다.

경로	화면	내용
/login	로그인	사번 + 비밀번호
/	대시보드	KPI 4종, 웨이퍼 카드 목록, 업로드

/review	리뷰 대기열	확신도 미달 건, 클래스 필터
/settings	설정	판정 임계값 (수석 이상)

웨이퍼 상세 팝업은 원본 / 히트맵 / 겹쳐보기 토글과, 1순위 승인 · 2순위 수정 · 수동 라벨링 세 가지 처리 경로를 담는다. 어떤 경로로 확정하더라도 AI의 원래 예측은 덮어쓰지 않고 검토 이력을 따로 쌓는다. 나중에 모델 성능을 되짚으려면 AI가 원래 무엇이라 했는지가 남아 있어야 하기 때문이다.

3. 실측 결과

LSWMD.pkl 원본 811,457건에서 학습 분할을 시드까지 재현해 측정했다. 균형 데이터 51,038건이 학습 로그와 정확히 일치해 재현이 맞음을 확인했다.

3.1 클래스별 성능 (검증 분할 10,208건)

클래스	정밀도	재현율	F1	표본
Edge-Ring	0.991	0.989	0.990	1,936
none	0.983	0.982	0.982	5,104
Center	0.965	0.977	0.971	859
Near-full	0.966	0.933	0.949	30
Random	0.958	0.931	0.944	173
Edge-Loc	0.910	0.936	0.923	1,038
Scratch	0.880	0.895	0.887	238
Loc	0.886	0.851	0.868	719
Donut	0.858	0.874	0.866	111

오류는 Loc ↔ Edge-Loc ↔ none 삼각형에 몰린다(Loc→Edge-Loc 36건, none→Edge-Loc 36건, none→Loc 33건). Loc은 정의상 Edge-Loc과 겹치고 결함이 옅으면 none과도 구분이 어렵다. 모델의 결함이라기보다 데이터셋 레이블 경계 자체의 모호성으로 보인다. Near-full은 표본이 30건뿐이라 수치를 그대로 믿기 어렵지만, 이 클래스는 밀도 규칙이 별도로 잡아주므로 AI 성능 의존도가 낮다.

3.2 판정 경로별 최종 정확도

경로	비율	최종 판정 정확도
AI 자동 통과 (AUTO_PASS)	94.86%	98.52%
규칙 확정 (Near-full)	0.20%	100%
엔지니어 검토 (REVIEW_NEEDED)	4.95%	56.63%

사람이 보지 않고 나가는 건이 95.05%이며, 그중 잘못 나가는 건은 전체의 1.40%다. 검토 대기 건에서 AI 1순위가 맞을 확률이 56.63%라는 점은, 확신도 임계값 0.80이 사람이 볼 값어치가 있는 건만 올리고 있음을 뜻한다.

3.3 학습에 쓰이지 않은 데이터에서의 정확도

균형 맞추기 과정에서 제외되어 학습도 검증도 한 번도 보지 않은 정상(none) 웨이퍼 20,000건에서 98.22%를 기록했다. 정상을 결함으로 오인한 상위는 Edge-Loc 150건, Loc 87건, Center 49건이다. 검증 분할과 합산(30,208건)하면 97.63%다.

보고 시 유의 — 편향이 없는 수치는 98.22% 하나뿐이다

96.4538%는 학습 중 최적 체크포인트를 고르는 데 사용된 분할에서 나온 값이라 낙관적으로 편향되어 있으며, 정상 클래스를 언더샘플링한 균형 데이터 기준이다. 발표 자료에는 "검증 정확도"로 명시하는 것이 정확하다. 학습 코드가 결함 샘플을 전부 학습·검증에 사용했기 때문에 완전히 독립적인 테스트 셋은 현재 존재하지 않는다.

3.4 이미지 경로의 영향 (실데이터 300건)

항목	값
두 경로 예측 일치	99.00% (297/300)
데이터 경로 정확도	97.33%
이미지 경로 정확도	96.67% (-0.66%p)
달라진 건	Loc→Edge-Loc, Loc→none ×2

이미지 편의성과 맞바꿀 만한 수준으로 판단했다. 정밀도가 중요한 경우를 위해 데이터 경로를 그대로 유지하고, 어느 경로로 들어왔는지 화면과 데이터베이스에 기록한다.

4. 설계 문서와 구현의 차이

1학기 산출 문서를 기준으로 구현하는 과정에서 확인된 차이다. 대부분 문서 쪽을 수정해야 하는 항목이며, 일부는 측정으로 설계가 반증된 경우다.

항목	문서	구현 / 확인된 사실
저밀도 규칙	밀도 5% 미만이면 none	실측으로 반증. 291건의 실제 결함을 덮음. 기본 비활성
판정 단계	문서마다 2단계 / 3단계로 상이	4단계로 통일, 정의되지 않은 구간 제거
손실 함수	기능명세서는 CrossEntropy	실제는 Focal Loss(γ =2). 플로우차트가 정확
weight_decay	1e-4	실제 0.05
Grad-CAM 변환	reshape_transform 필요	출력이 4D NCHW라 불필요
정확도	약 95.7%	체크포인트 기록 96.45% (검증 기준)
데이터 규모	학습 172,950건	실제 균형 데이터 51,038건
테스트 분할	Train 80 / Val 10 / Test 10	실제는 8:2 분할만. 테스트 셋 없음
입력 형식	PNG/JPG 사진 업로드	모델은 0/1/2 맵 학습. 렌더링 이미지만 가능
ERD	7개 테이블, 사용자 테이블 없음	Vision 5개 구현 + users 추가
대시보드	app.py는 React	실제는 Streamlit (센서 프로젝트 소속)
API 명세서	내용이 타 프로젝트로 덮어쓰기됨	복구 필요. 현재는 플로우차트 §5가 유일한 API 목록

5. 남은 작업

단계	내용	선행 조건
Phase 5	Docker Compose(api/db/nginx), HTTPS, 배포	서버 확보
Phase 6	재학습 루프 (정답 500건 트리거)	Phase 5 완료
문서 정비	API 명세서 복구, 위 4장 항목 반영	-
테스트 셋	train/val/test 3분할 재학습	Phase 6과 병행

서버는 아직 확정하지 않았다. 권장 사양은 4 vCPU / 8GB RAM / 50GB이며, 학교 또는 학과의 캡스톤 서버·클라우드 크레딧 지원을 먼저 확인한 뒤, 없을 경우 무료 티어(Oracle Cloud, 춘천 리전 ARM 4코어/24GB) 또는 고정 요금제(AWS Lightsail 서울, 월 $24~44)를 검토한다. Phase 5의 Docker 구성 자체는 서버 없이 로컬에서 작성·검증할 수 있다.

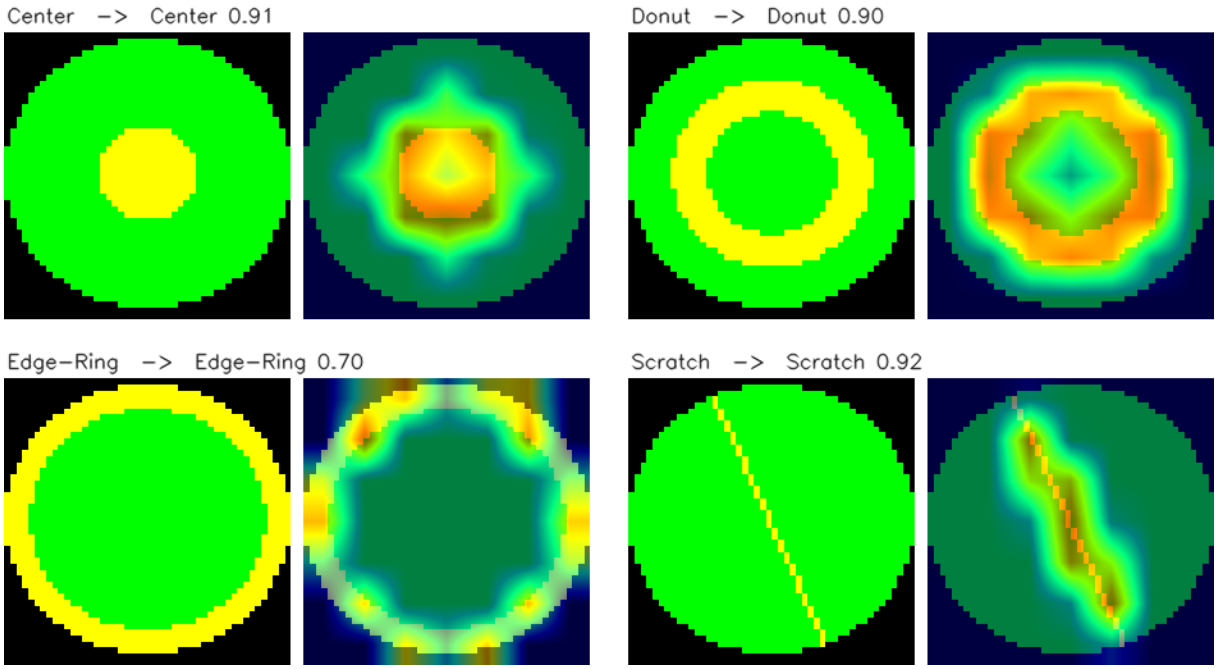
6. 한계와 유의사항

- 완전히 독립적인 테스트 셋이 없다. 학습 코드가 결함 샘플을 전부 소진해, 편향 없는 추정치는 미사용 정상 데이터 20,000건의 98.22%뿐이다.

- 확신도 임계값 0.80은 기존 코드에서 가져온 값이다. 검토 대기 건의 AI 정답률 56.63%로 타당성을 뒷받침했으나, 비용 관점의 최적화는 수행하지 않았다.
- 이미지 경로는 학습과 동일한 색 체계로 렌더링된 맵에 한해 동작한다. 검사 장비마다 색 구성이 다르면 별도 매핑이 필요하다.
- Sensor AI 파이프라인(SECOM, Isolation Forest)과 통합 리포트는 이번 학기 범위에서 제외했다. ERD의 관련 테이블도 구현하지 않았다.
- 부하 시험을 수행하지 않았다. 동시 접속자 수에 따른 응답 시간은 배포 후 측정이 필요하다.

부록 A. Grad-CAM 히트맵 예시

각 행의 왼쪽이 모델이 실제로 입력받은 웨이퍼 맵, 오른쪽이 히트맵을 겹친 결과다. 붉은 영역이 AI가 판단 근거로 삼은 위치다.

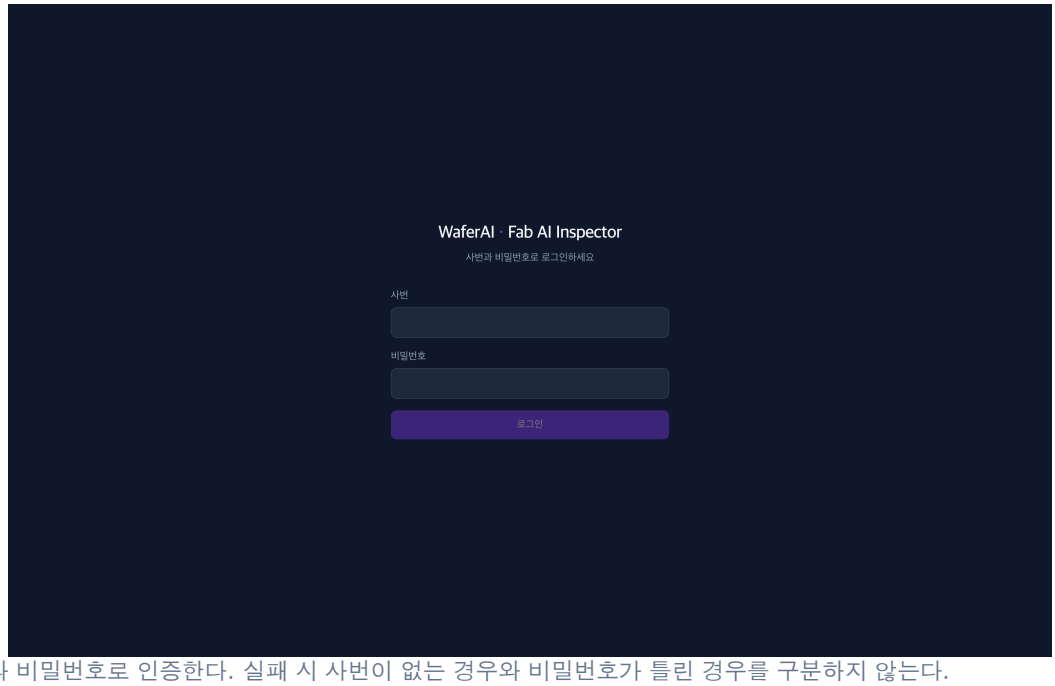


합성 웨이퍼 4종 (Center · Donut · Edge-Ring · Scratch)

부록 B. 화면 구성

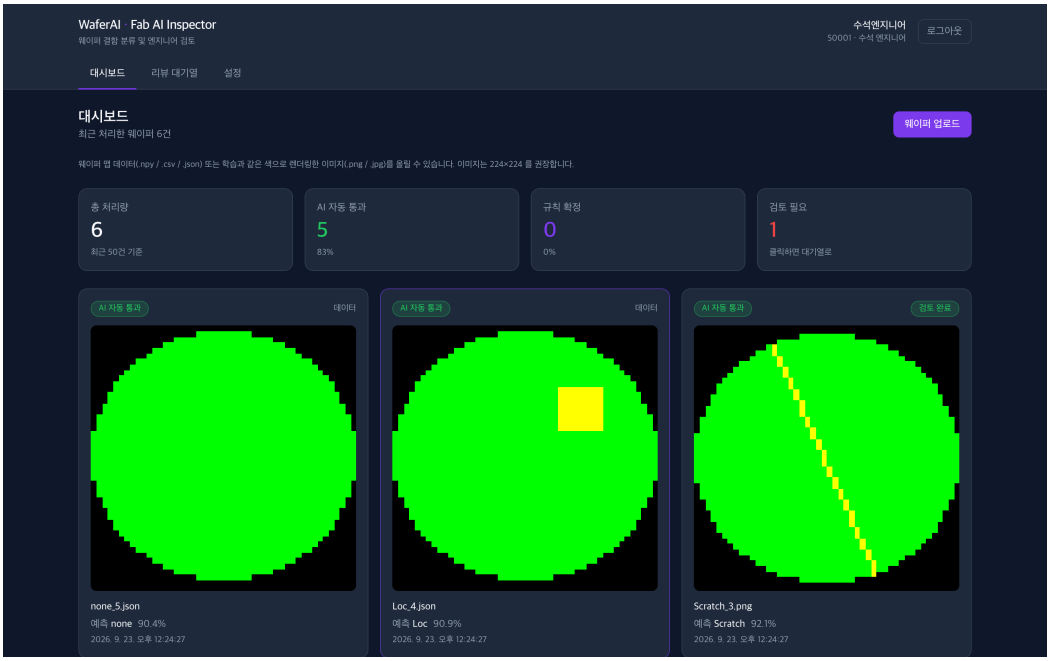
실제 동작 화면이다. 로그인 후 메인 대시보드가 나오고, 헤더 아래 탭으로 다른 화면으로 이동한다. 등급에 따라 탭 구성이 달라진다.

화면 1. 로그인



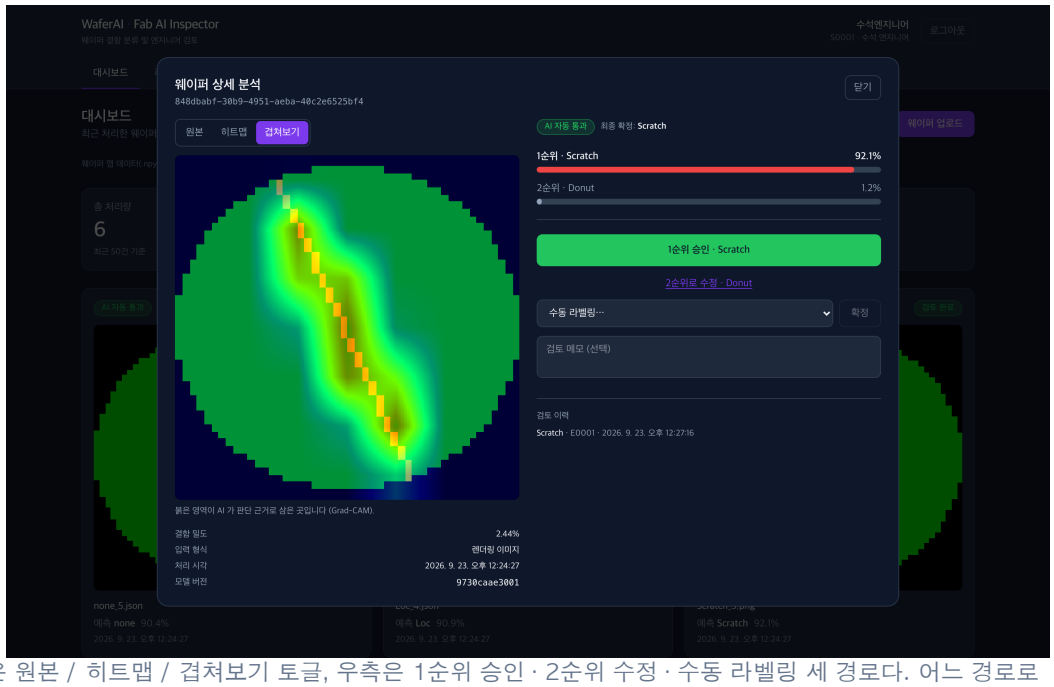
사번과 비밀번호로 인증한다. 실패 시 사번이 없는 경우와 비밀번호가 틀린 경우를 구분하지 않는다.

화면 2. 메인 대시보드



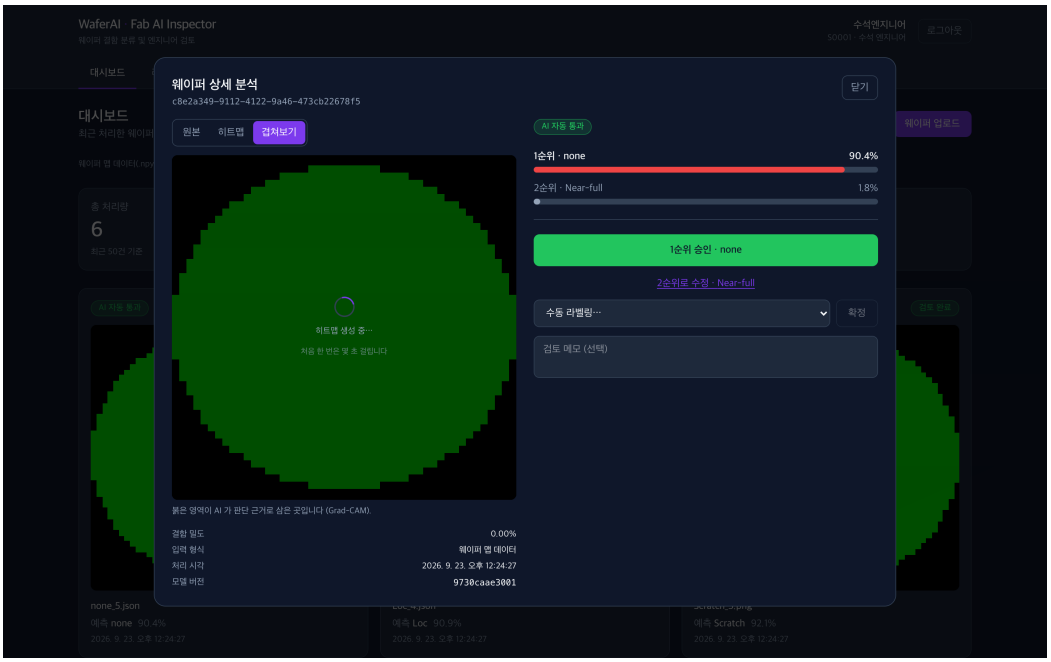
상단 KPI 4종으로 처리 현황을 요약하고, 아래 카드 목록에서 개별 웨이퍼로 들어간다. 목록 썸네일은 모델을 거치지 않는 원본 렌더링이라 목록을 여는 비용이 낮다.

화면 3. 웨이퍼 상세 — Grad-CAM



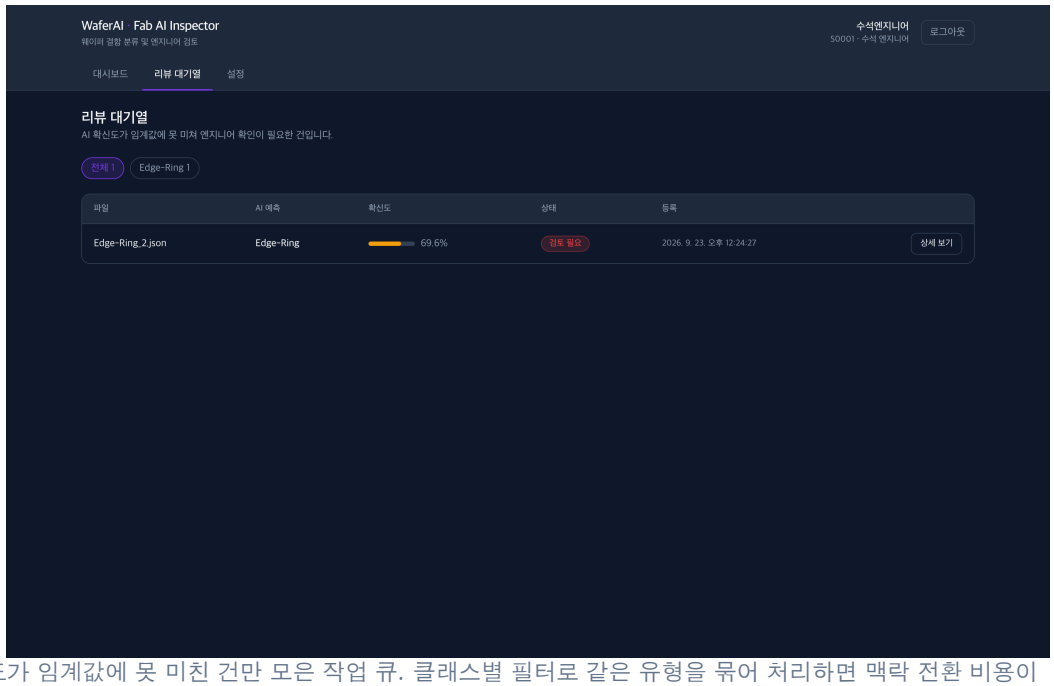
좌측은 원본 / 히트맵 / 겹쳐보기 토글, 우측은 1순위 승인 · 2순위 수정 · 수동 라벨링 세 경로다. 어느 경로로 확정해도 AI 원래 예측은 보존되고 검토 이력이 따로 쌓인다.

화면 3-1. 히트맵 생성 중



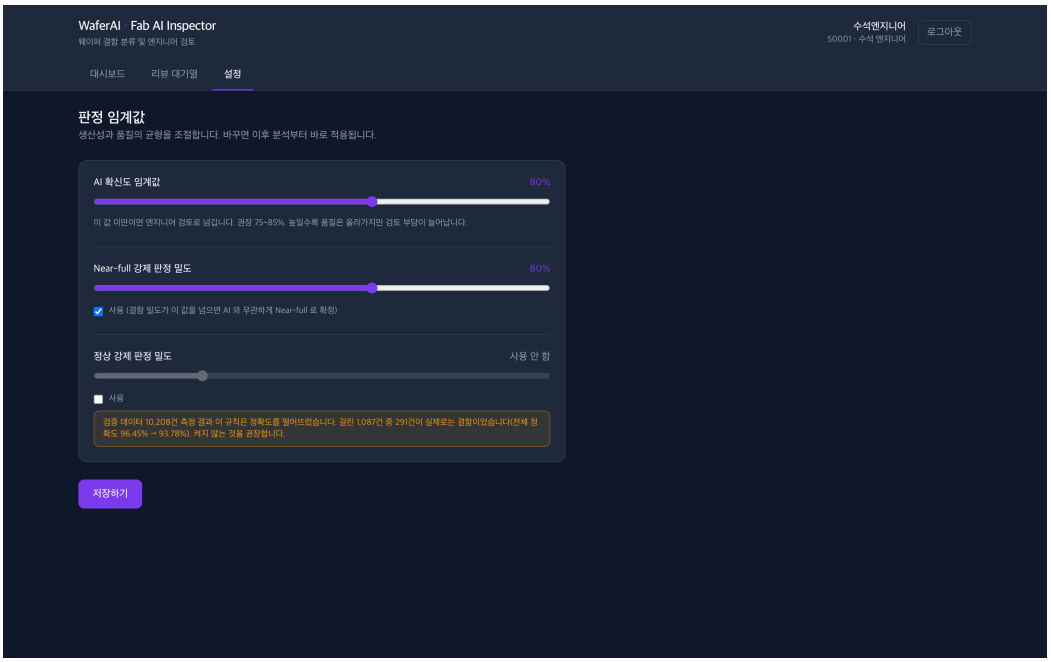
히트맵은 지연 생성이라 캐시가 없는 첫 요청에서 모델 적재와 역전파가 함께 일어난다. 몇 초가 걸리므로 진행 상태를 표시한다.

화면 4. 리뷰 대기열



확신도가 임계값에 못 미친 건만 모은 작업 큐. 클래스별 필터로 같은 유형을 묶어 처리하면 맥락 전환 비용이 줄어든다.

화면 5. 판정 임계값 설정



수석 엔지니어 이상만 접근한다. 저밀도 규칙에는 본문 2.3 의 측정 근거를 경고로 띄워, 이후에 무심코 켜지 않도록 했다.